- Final Project
  - Final Project Proposal
    - 1. Abstract
    - 2. Motivation
    - 3. System Block Diagram
    - 4. Design Sketches
    - 5. Software Requirements Specification (SRS)
    - 6. Hardware Requirements Specification (HRS)
    - 7. Bill of Materials (BOM)
    - 8. Final Demo Goals
    - 9. Sprint Planning
  - Sprint Review #1
    - Last week's progress
    - Current state of project
    - Next week's plan
  - Sprint Review #2
    - Last week's progress
    - Current state of project
    - Next week's plan
  - MVP Demo
  - Final Report
    - 1. Video
    - 2. Images
    - 3. Results
      - 3.1 Software Requirements Specification (SRS) Results
      - 3.2 Hardware Requirements Specification (HRS) Results
    - 4. Conclusion
  - References

Review the assignment due date

Final Project

Team Number: 10

Team Name: Gimbalers

Team Member Name	Email Address
Cindy Liu	[cinl@seas.upenn.edu]
Mike Song	[chengyus@seas.upenn.edu]
Justin Yu	[justinyu@seas.upenn.edu]

GitHub Repository URL: https://github.com/upenn-embedded/final-project-s26-t10

GitHub Pages Website URL: [for final submission]*

Final Project Proposal

1. Abstract

This project is the design and implementation of a 3-axis camera gimbal system that uses servos to provide active stabalization for a camera. It uses an Inertial Measurement Unit (IMU) to monitor angular rotation as an input. Feedback is processed by an ATMega328PB. The microcontroller will output PWM signals, corrected by a PID control algorithm. Finally, the entire system will be powered by USB-C.

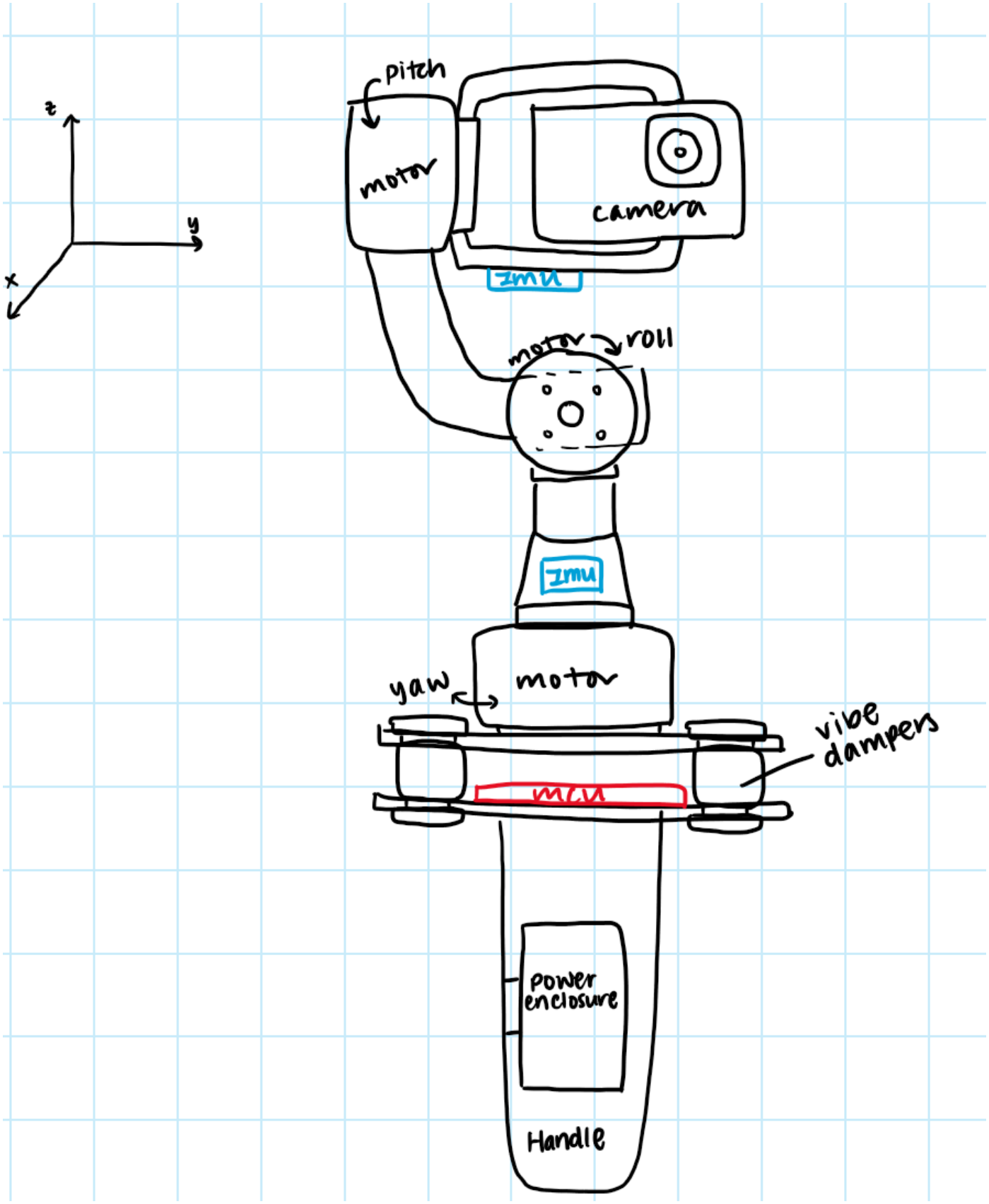
2. Motivation

Conventional camera gimbals are expensive, usually upwards of $200. Furthermore, they are usually designed specifically for industry grade cameras, leaving much to be desired for amateur videographers. This project gives an affordable option for those looking to try recording videos wth stabalization. By utilizing the widespread USB-C connector for power, this device can be used in many environments with a variety of power sources.

3. System Block Diagram

![alt text](Images/System Block Diagram.drawio.png)

4. Design Sketches



5. Software Requirements Specification (SRS)

5.1 Definitions, Abbreviations

(IMU) - Inertial Measurement Device, gives linear acceleration and angular rate data
(Angular Rate) - degrees per second, as given by IMU (PID controller) Proportional-Integral-Derivative controller. Uses feedback to hit a target while reducing overshoot, in

this case to stabalize the axis after rotation. Uses integral and derivative of error to adjust output. (PWM) - pulse width modulation, produces a square wave output from the microcontroller to serve as an input to the servo.

5.2 Functionality

ID	Description
SRS-01	The IMU 3-axis angular rate will be measured with 16-bit depth every 100 milliseconds ±10 milliseconds.
SRS-02	The PID controller will recieve new inputs and produce PWM outputs for each servo every 100 milliseconds ±10 milliseconds, based on estimated error with an accumulator variable that sums IMU's angular rate every polling cycle.
SRS-03	The PWM Duty cycle that is output to the servos will be updated every 100 milliseconds ±10 milliseconds.
SRS-04	The gimbal will read the input of an "enable" button so it will stop stabalizing if the button is pressed.
SRS-05	The device can be reset to zeroed positions with the press of the "zero" button.
SRS-06	The device can be manually zeroed and the position can then be remembered with a long press of the "zero" button.
SRS-07	The user shall be able to determine the state of the device operation by viewing the status LED.

6. Hardware Requirements Specification (HRS)

6.1 Definitions, Abbreviations

MCU - Microcontroller Unit, compact integrated circuit that processes inputs and manages dedicated outputs and tasks. Camera Plate - Platform for GoPro/similar sized camera to rest on, controlled on 3 axes by the servos. Zeroing - Setting the default position for the camera plate, manually done by aligning the camera plate with the handle.

6.2 Functionality

ID	Description
HRS-01	USB-C PD module should deliver 20V output from an USB-C input.
HRS-02	Buck Converter Module should deliver stable 5V output.
HRS-03	ATmega328PB will function as the microcontroller for the device and also deliver 3.3V.
HRS-04	2 IMUs interfaced via I2C will deliver acceleration and position data to the MCU.
HRS-05	3 servos will control the pitch, roll, and yaw of the camera plate.
HRS-06	Two buttons that allows users to enable the gimbal function and zero the servos.
HRS-07	Servos combined with IMU inputs stabilize movement of camera plate in three degrees of freedom.
HRS-08	Servos combined with the IMU inputs can "lock" the camera onto a specific location/direction.

7. Bill of Materials (BOM)

https://docs.google.com/spreadsheets/d/1yj54xOVig_wChPn7wCOqRE96ZmyI3QeAy7MwsV0c4z8/edit?gid=2071228825#gid=2071228825

8. Final Demo Goals

Demonstrate a 3-axis (pitch and roll) active stabilization. Able to keep camera platform stable despite rapid movements. The system shall read angular velocity from an IMU, compute a stable angle estimate, and drive three servos. Independent PID controllers on each axis shall correct for angular error at 10Hz.

9. Sprint Planning

Milestone	Functionality Achieved	Distribution of Work
Sprint #1	Using an I2C Library, stabilize pitch, create handle and first servo mount	Mike will CAD the handle and servo mount prototype. Justin and Cindy will implement and tune PID for one axis.
Sprint #2	Move away from using I2C library, include second servo and begin tuning both axes in tandem, then moving on to include the third and last servo.	Mike will design and integrate the complete 3 servo mount and validate mechanical balance on all axes. Justin and Cindy will work on integration between the 3 axes and the PID loop.
MVP Demo	All axes stabilizing with servo actuation. Complementary filter producing stable angle estimates on all axes. PID visibly rejecting hand-induced disturbances. Mechanical assembly fully integrated.	Mike ensures full mechanical assembly is complete and camera platform is balanced. Cindy integrates both PID loops and validates stable closed-loop behavior on pitch and roll. Justin validates I2C driver and verifies clean IMU data.
Final Demo	Custom I2C driver, complementary filter, PID controllers, all written from scratch in register-level C. System visibly stabilizes camera footage under moderate hand disturbance at 10Hz.	Mike finalizes mechanical assembly, performs cable management, and captures demo footage. Justin and Cindy tunes PID gains on all axes, implements deadband, and optimizes control loop timing.

This is the end of the Project Proposal section. The remaining sections will be filled out based on the milestone schedule.

Sprint Review #1

Last week's progress

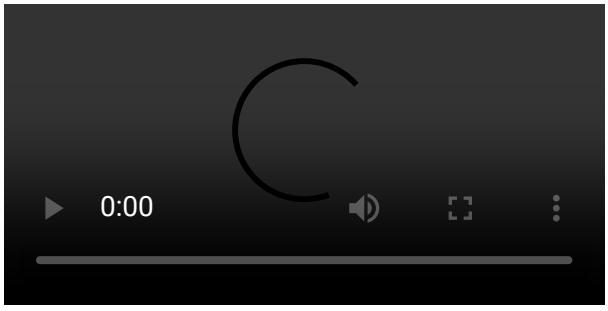
In the past week, we started working on both the mechanical structure of the gimbal and the software for the IMU reading and servo control. We were able to get a preliminary assembly in CAD of the mates for each axis of rotation, as well as model the servo motors and camera frame. On the software side, we were able to use I2C to read the angle from the IMU, compute servo correction position, and generate an output PWM to stablize the system. Finally, we have also placed the order for our remaining parts.

Current state of project

Code file: see main.C

CAD: ![alt text](Images/CAD design sprint1.png)

Video of motor stabalization in response to IMU:



Link to video:
https://drive.google.com/file/d/1zUX_p3jPe_XTrYJxmcY68znGZ1ic0DGx/view?usp=drive_link

Being able to get the motor to smoothly respond to rotation in one axis is a major step and provides a solid foundation for building on top of that. We foresee a lot more challenges in integrating PID into a multi-axis system since the axis and position will affect one another, but our current state is enough to ensure at least minimum functionality.

Next week's plan

Finish the CAD assembly and start 3D printing the parts at RPL. The CAD should take half a week and printing the parts may take longer, up to 1 week. (Mike)

Refine the algorithm for one-axis PID, and add in second IMU and servo. Expect to take up to 1.5 weeks. Start experimenting with power delivery with sample PD board from Detkin before actual power parts arrive, take around 1 week. (Justin and Cindy)

Sprint Review #2

Last week's progress

In the past week, we have made progress on both the mechanical assembly and the PID control. The parts of the assembly that were CADed last week have been 3d-printed such as the go-pro mount and its bracket, and we are testing to see if the parts fit. New parts were added to the CAD assembly, including the board mount and the handle. Some of our parts have also arrived, including a servo motor of our specified model that we used for testing the PID algorithm.

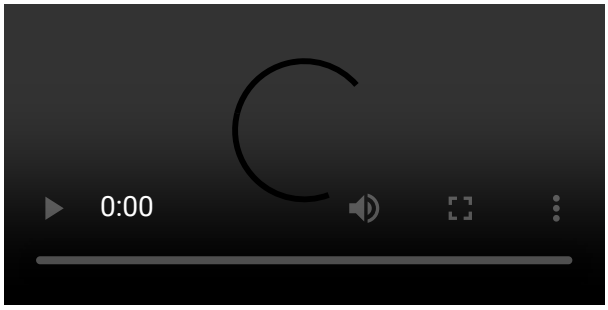
On the software side, we fine-tuned the kp parameter and dt of our PID control. Now the servo responds at a faster rate and the error between the IMU's rotation and the servo's rotation has been decreased. We then assembled the IMU, servo, and bracket together to create a 1-axis prototype.

Current state of project

Code file: see main.C

CAD: ![alt text](Images/Sprint Review2 CAD.png)

Video of 1-axis prototype:



Link to video:
https://drive.google.com/file/d/1BJ6jZqnTRZo793Z51MHxtKvuvLm101JA/view?usp=drive_link

From the video, we see that the angle of the servo (where the go-pro camera mount will go) remains around the same angle relative to the ground.

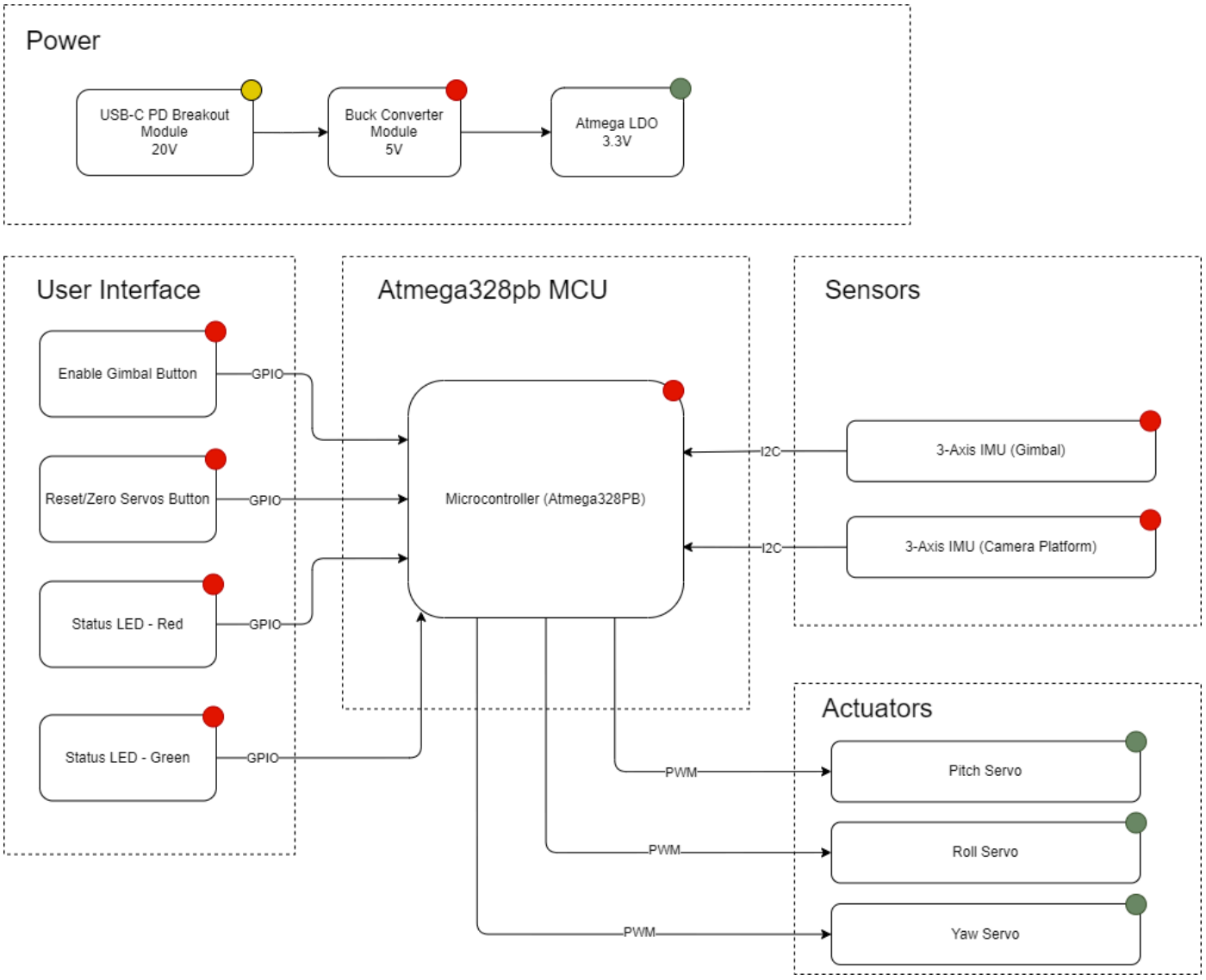
Next week's plan

Finalize the CAD, print all the parts, and build a full assembly. This will allow us to know which parts need to be remeasured and fixed for the final print. (Assigned to Cindy and Mike, finish by MVP demo day). On the software side, the first goal is to integrate the other two servos so that we have a kp parameter tuned for three servos at once. (Assigned to Justin, finish by MVP demo day). If time permits, we can start adding I and D elements into the code to decrease error and oscillation.

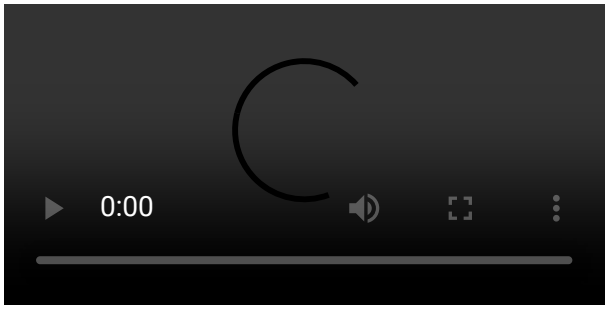
MVP Demo

The plan for the system block diagram has not changed significantly since the project proposal. For our MVP, we have the I2C lines and PWM lines integrated. The MCU is able to communicate with sensors and actuators consistently, taking the data from the

IMU and servos and implementing PID control.



Our device currently features two axis stabilization, as demonstrated by this video:



Link: https://drive.google.com/file/d/1pYFkpmA01hFtK_cwW8RbuCtw5umUj9uW/view?usp=sharing

The software implementation can be found in mvp.c on our gitHub. The software maintains a consistent sampling period (dt), in which it polls the IMU readings from. It then multiplies the rotation rate by dt to calculate the new angle. This angle is then used to calculate the error term (using KP paramter), the integral, and the derivative terms. Then, a PWM is sent to both servos to move the arms. Beyond the control

algorithm, the rest of our software includes UART and I2C libraries, as well as gyro calibration and other setup functions.

For our project software requirements:

We met/are close to meeting the following:

SRS-01: The IMU 3-axis angular rate will be measured with 16-bit depth every 100 milliseconds. Currently we are actively using only 2 of the angular rates.

SRS-02: The PID controller will receive new inputs and produce PWM outputs for each servo every 100ms, based on estimated error with an accumulator varaible that sums IMU's angualr rate every polling cycle. This has been achieved for our two servos.

SRS-03: The PWM duty cycle that is outputted to the servos will be updated every 100ms. This also been achieved for our two servos.

SRS-05: The device can be reset to zeroed positions with the press of the "zero" button. The reset feature is currently implemented in software; we still need to add a hardware button and input capture.

We still need to work on the following:

SRS-04: The gimbal will read the input of an "enable" button so it will stop stabailizing if the button is pressed.

SRS-06: The device can be manually zeroed and the position can then be remembered with a long press of the "zero" button.

SRS-07: The user shall be able to determine the state of the device operation by viewing the status LED.

Throughout testing, we collect data on angle, error, and servo ticks using the serial terminal, example below:

```
Calibrating (hold still)...

Bias Y: -2979/100
Bias Z: 38338/100
dt_us,pitch_x100,roll_x100,s1,s2
8640,-35,64,3012,2982
8640,-107,238,3038,2925
8704,-127,452,3043,2863

8640,-139,727,3048,2779

8704,-145,849,3051,2742

8640,-142,890,3050,2728

8704,-134,890,3047,2726
8640,-120,896,3043,2722

8704,-114,900,3041,2718
8704,-112,898,3041,2716

8704,-96,898,3036,2713

8704,-87,904,3033,2708
8704,314,913,2887,2702
```

For our project hardware requirements: We have met/are close to meeting the following:

HRS-03: ATmega328PB will function as the microcontroller for the device and also deliver 3.3V.

HRS-04: 2 IMUs interfaced via I2C will deliver acceleration and position data to the MCU.

HRS-05: 3 servos will control the pitch, roll, and yaw of the camera plate.

HRS-07: Servos combined with IMU inputs stabilize movement of camera plate in three degrees of freedom.

We still need to work on the following power-related hardware goals. Since the PD board has not arrived yet, we are powering the MVP through a power supply instead of a portable usb PD + buck converter.

HRS-01: USB-C PD module should deliver 20V output from an USB-C input.

HRS-02: Buck Converter Module should deliver stable 5V output.

We still need to work on the remaining UI related hardware goals:

HRS-06: Two buttons that allows users to enable the gimbal function and zero the servos.

HRS-08: Servos combined with the IMU inputs can "lock" the camera onto a specific location/direction.

Our plan moving forward before demo is:

1. Improve tuning of the first two servos and add the second IMU.
2. Mechanically assemble the base plates and handles
3. Add in the third servo and tune PID
4. Add in buttons for user reset and enable
5. Fix and clean up the wiring.
6. If time permits, implement the power module.

Thus, the remaining portions that will make our project complete include assembling the mechanical handle (which we already have the CAD for), programming the 3rd servo, implementing the buttons, and wiring the power distribution.

The riskiest part remaining of our project is being able to achieve stabilization on all three axes with minimal jittering. Currently, we see a lot of jittering even with just two axes, and this poses the risk of damaging the mechanical assembly (we've had screws come lose from the jittering before). The risk is that after adding in the third axes, uncontrolled oscillations coming from all three servos would cause the entire device to not only be unstable, but also amplifier the error in our rotations. We plan to de-risk this by strengthening the mechanical assembly, re-tune the first two axes, and improve accuracy by adding in a second IMU.

Final Report

Don't forget to make the GitHub pages public website! If you've never made a GitHub pages website before, you can follow this webpage (though, substitute your final project repository for the GitHub username one in the quickstart guide): https://docs.github.com/en/pages/quickstart

1. Video

2. Images

3. Results

3.1 Software Requirements Specification (SRS) Results

ID	Description	Validation Outcome
SRS-01	The IMU 3-axis acceleration will be measured with 16-bit depth every 100 milliseconds +/-10 milliseconds.	Confirmed, logged output from the MCU is saved to "validation" folder in GitHub repository.

3.2 Hardware Requirements Specification (HRS) Results

ID	Description	Validation Outcome
HRS-01	A distance sensor shall be used for obstacle detection. The sensor shall detect obstacles at a maximum distance of at least 10 cm.	Confirmed, sensed obstacles up to 15cm. Video in "validation" folder, shows tape measure and logged output to terminal.

4. Conclusion

References